



DESIGN AND ANALYSIS OF AN ENERGY-EFFICIENT PROCESSOR FOR AN INDUSTRIAL MONITORING SYSTEM

Department of Computer Science and Engineering

Field	Details
Course Code & Name	CSA1203 - Computer Architecture
Assignment Title	Design and Analysis of an Energy-Efficient Processor and Hierarchical Memory System
Course Outcomes (CO)	CO1: Analyze functional units of a computer system (CPU, memory, buses, I/O), instruction execution cycles, addressing modes, and performance metrics (CPI, clock cycles, benchmarking). CO3: Design processor organization - pipelined datapath, hazard types (data, control, structural), and superscalar techniques (out-of-order execution, speculative execution, SMT). CO4: Implement hierarchical memory systems - cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM).
Bloom's Taxonomy Level	L3 - Apply; L4 - Analyze
SDG Mapping	SDG 7 - Affordable and Clean Energy SDG 9 - Industry, Innovation and Infrastructure SDG 12 - Responsible Consumption and Production

Field	Details
Submitted By	Kanishk C
Registration Number	192511278
Institution	SIMATS Engineering (Saveetha Institute of Medical and Technical Sciences)

Problem Statement

A smart manufacturing company plans to develop an energy-efficient processor for an industrial monitoring system. The processor must handle sensor data acquisition and real-time processing requirements reliably while keeping power consumption low. This assignment analyzes the functional units of such a processor (CPU, memory, buses, and I/O components), applies the instruction execution cycle and addressing modes for efficient instruction processing, evaluates performance using CPI, clock cycles, execution time and benchmarking, designs a suitable pipelined datapath to improve instruction throughput, analyzes data, control and structural hazards, applies forwarding, stalling and branch-prediction techniques to reduce their impact, compares superscalar processing, out-of-order execution, speculative execution and simultaneous multithreading (SMT), designs a hierarchical memory system using L1, L2 and L3 caches with virtual memory, compares SRAM, DRAM, NAND Flash, MRAM and RRAM on latency, bandwidth, throughput and power, and finally recommends and justifies an optimized processor and memory organization for the system - explaining how the proposed design supports SDG 7, SDG 9 and SDG 12.

1. Introduction

Industrial monitoring systems continuously collect readings from sensors such as temperature, pressure, vibration, and flow sensors, and must process this data in real time to detect faults, trigger alerts, and control actuators. Because these systems are frequently deployed in large numbers on a factory floor and often run on battery or limited power budgets, the processor at their core must strike a balance between three competing requirements: sufficient computational throughput to process sensor streams without lag, predictable and low-latency response for real-time control, and minimal energy consumption to keep operating costs and heat output low.

This assignment designs and analyzes a processor and memory system for exactly this scenario. It works through the functional units of the CPU, the instruction execution cycle and addressing modes used to process sensor data efficiently, standard performance metrics (CPI, clock cycles, execution time, and benchmarking), a pipelined datapath with hazard analysis, a comparison of advanced execution techniques (superscalar, out-of-order, speculative execution, and SMT), and a hierarchical memory system built from L1/L2/L3 caches, virtual memory, and a comparison of modern memory technologies. The assignment closes with a consolidated engineering recommendation and a reflection connecting the design choices to the United Nations Sustainable Development Goals (SDG) 7, 9, and 12.

2. Functional Units of the Processor (CO1)

A general-purpose computer system, and by extension our industrial-monitoring processor, is organized around five functional units that cooperate over a shared bus: the CPU (control unit and ALU), the register file, the memory hierarchy, the system bus, and the I/O subsystem. Figure 1 shows how these units are organized for the industrial-monitoring use case, with sensors and actuators attached through an I/O interface and DMA controller so that bulk sensor data does not have to pass through the CPU one word at a time.

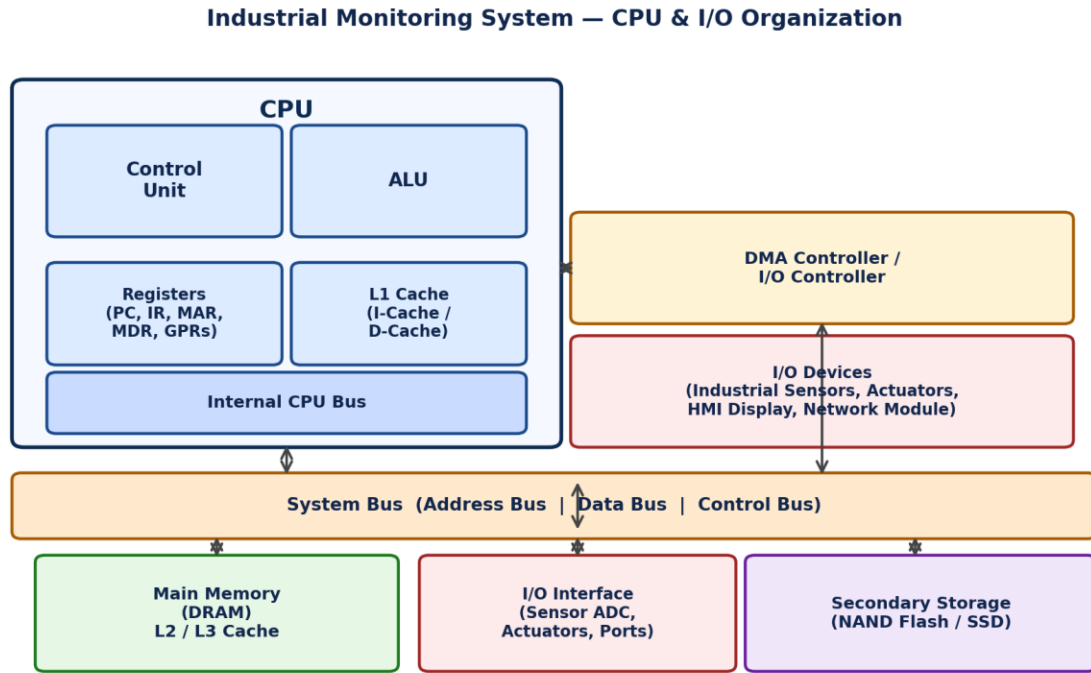


Figure 1: CPU and I/O organization of the proposed industrial-monitoring processor.

2.1 Control Unit

The control unit (CU) generates the timing and control signals that coordinate every other functional unit. It reads the opcode from the instruction register, decides which micro-operations are required (register read, ALU operation, memory access, register write-back), and asserts the corresponding control lines during each clock cycle. In this design the CU is hardwired rather than microprogrammed, since a fixed, small industrial-control instruction set does not need the flexibility of microcode and a hardwired CU gives lower latency and lower power per instruction - both important for an embedded monitoring node.

2.2 Arithmetic and Logic Unit (ALU)

The ALU performs the arithmetic (add, subtract, multiply, compare) and logic (AND, OR, XOR, shift) operations needed to convert raw sensor readings into engineering units, run threshold checks, and compute simple filters (e.g., a moving average of a vibration signal). A dedicated fixed-point/simple floating-point ALU is sufficient here; full IEEE-754 double precision hardware would add power and area that this application does not need.

2.3 Registers

The register file includes the Program Counter (PC), Instruction Register (IR), Memory Address Register (MAR), Memory Data Register (MDR), a set of general-purpose registers (GPRs) for holding intermediate sensor values, and status/flag registers (zero, carry, overflow, negative) used by conditional branches that implement threshold checks such as "if temperature > limit, raise alarm."

2.4 Bus Structures

Three logical buses connect the functional units: the address bus (carries the memory or I/O address being accessed), the data bus (carries the actual data word), and the control bus (carries read/write, interrupt, and clock/handshake signals). A single shared system bus keeps the design simple and low-power, which fits an embedded monitoring node; a split Harvard-style instruction/data bus is used internally between the CPU and L1 cache to allow simultaneous instruction fetch and data access, which materially helps pipelining (Section 5).

2.5 I/O Components

Sensors (temperature, pressure, vibration, flow) and actuators (valves, motors, alarms) are attached through an I/O interface containing analog-to-digital converters and signal conditioning, and a DMA controller that can move a block of sensor samples directly into memory without CPU intervention, freeing the CPU to perform analysis rather than data shuffling. This is important because a real industrial line may have dozens of sensors sampled every few milliseconds; polling each one individually from the CPU would waste cycles and energy.

3. Instruction Execution Cycle and Addressing Modes (CO1)

3.1 The Instruction Execution Cycle

Every instruction executed by the processor - whether it is reading a sensor value, comparing it to a threshold, or writing a control output - passes through the same basic cycle:

- Fetch: the instruction addressed by the PC is read from instruction memory / I-cache into the IR, and the PC is incremented.
- Decode: the control unit decodes the opcode and identifies the operands and addressing modes involved.

- **Operand Fetch:** register operands are read from the register file, or effective addresses are calculated for memory operands.
- **Execute:** the ALU performs the required arithmetic/logic operation, or the control unit resolves a branch condition.
- **Memory Access:** if the instruction is a load or store, the data memory / D-cache is accessed.
- **Write-Back:** the result is written back to the destination register (or memory), completing the instruction.

For example, reading a temperature sensor value stored at a fixed memory-mapped I/O address, comparing it against a limit held in a register, and branching to an alarm routine if the limit is exceeded, exercises all six stages: fetch the load instruction, decode it, compute the sensor's address, read the value into a register, compare it (execute), and conditionally branch (write-back of the new PC).

3.2 Addressing Modes

Addressing modes determine how the effective address of an operand is calculated. The table below lists the modes used in this design and how each one is applied to industrial sensor-data processing.

Addressing Mode	How the Operand Is Located	Use in Sensor-Data Processing
Immediate	Operand value is encoded directly in the instruction	Loading a fixed threshold constant, e.g. MOV R1, #85 (temperature limit)
Register	Operand is held in a CPU register	Comparing two already-loaded sensor readings, e.g. CMP R1, R2
Direct	Instruction contains the full memory address	Reading a sensor value from a fixed memory-mapped address
Register Indirect	Register holds the address of the operand	Walking through a buffer of samples via a pointer register
Indexed	Base address + index register (with optional scaling)	Accessing element <i>i</i> of an array of samples from <i>N</i> sensors
Base + Offset (Displacement)	Base register + constant offset	Accessing a specific field inside a sensor's control/status register block
Auto-increment / Auto-decrement	Register used as pointer, then automatically adjusted	Streaming successive ADC samples into a circular buffer

Register-indirect and indexed addressing dominate in this application because sensor samples are naturally stored as arrays/buffers (one buffer per channel, refreshed by DMA); direct and immediate addressing are used for fixed configuration constants such as calibration offsets and alarm thresholds.

4. Performance Evaluation - CPI, Clock Cycles, Execution Time and Benchmarking (CO1)

Processor performance is quantified using three related metrics: the number of clock cycles consumed, the average Cycles Per Instruction (CPI), and the resulting execution time, given by:

$$\text{Execution Time} = (\text{Instruction Count} \times \text{CPI}) / \text{Clock Rate}$$

To make this concrete, assume the processor runs at a clock rate of 1 GHz and executes a representative sensor-processing routine (read sample, scale it, compare to threshold, and conditionally branch) whose instruction mix is shown below.

Instruction Type	Instruction Count	Cycles Per Instruction (CPI)	Total Cycles
Load / Store (memory access)	3,000	4	12,000
ALU (add, sub, compare, shift)	5,000	1	5,000
Branch (conditional / unconditional)	1,500	3	4,500
Multiply / Divide (scaling & filtering)	500	6	3,000

Total instruction count = 3,000 + 5,000 + 1,500 + 500 = 10,000 instructions.

Total clock cycles = 12,000 + 5,000 + 4,500 + 3,000 = 24,500 cycles.

Average CPI = Total Cycles / Instruction Count = 24,500 / 10,000 = 2.45 cycles/instruction.

Execution Time = Total Cycles / Clock Rate = 24,500 / (1 x 10⁹ Hz) = 24.5 microseconds per sensor-processing routine.

This means that, without pipelining, the processor could evaluate roughly 1 / 24.5 microsec approx 40,800 such sensor routines per second on a single core - comfortably fast enough for a monitoring loop sampled every few milliseconds, but with headroom that pipelining and caching (Sections 5-

7) can convert into either faster response or lower clock frequency (and therefore lower power) for the same throughput.

4.1 Benchmarking

Beyond a single hand-traced routine, real processor selection relies on benchmarking - running standardized or application-representative workloads and comparing metrics such as MIPS (millions of instructions per second), MFLOPS (millions of floating-point operations per second, if applicable), and, for embedded/industrial parts, EEMBC CoreMark or similar embedded benchmarks that better reflect control-loop and signal-processing workloads than desktop benchmarks such as SPEC. For this design, CoreMark-per-milliwatt is the most relevant figure of merit, since it captures both throughput and the energy-efficiency goal (SDG 7) simultaneously, rather than throughput alone.

5. Pipelined Datapath Design and Hazard Analysis (CO3)

To raise instruction throughput beyond the single-cycle-per-instruction-group model of Section 4, the processor uses a classic 5-stage pipeline: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB). Each stage is handled by a different piece of hardware, so up to five instructions can be in flight simultaneously, one per stage, increasing throughput even though any single instruction still takes five cycles to complete.

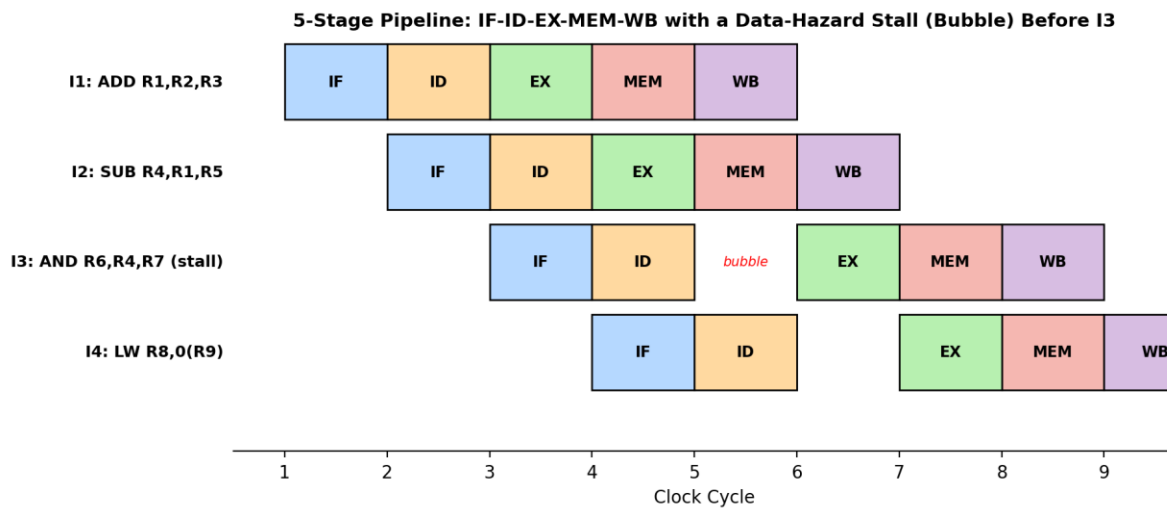


Figure 2: Five-stage pipeline timeline. I3 must stall (insert a bubble) for one cycle because it needs R4, which I2 does not produce until the end of its EX stage.

5.1 Pipeline Hazards

- **Structural Hazard** - two instructions need the same hardware resource in the same cycle (e.g., a single memory port needed by an instruction fetch and a data load at once). Solved here by a split I-cache/D-cache (Harvard-style) so fetch and memory-access stages never contend for the same port.
- **Data Hazard** - an instruction needs a value that a previous, still-in-flight instruction has not yet written back (as illustrated by I2/I3 in Figure 2, where I3's AND needs R4 from I2's SUB).
- **Control Hazard** - a branch's outcome is not known until its EX stage, but the pipeline has already fetched the next 1-2 instructions speculatively; if the branch is taken, those fetched instructions must be discarded (a "pipeline flush").

5.2 Hazard-Mitigation Techniques

Forwarding (bypassing): the ALU result from the EX or MEM stage of one instruction is routed directly to the EX stage input of a dependent instruction, without waiting for it to be written back to the register file. This removes most data hazards without stalling; only a load-use hazard (a value loaded from memory used by the very next instruction) still needs one stall cycle, because the loaded value is not available until the end of MEM.

Stalling (pipeline interlock): when forwarding alone cannot resolve a hazard, the control unit inserts a bubble - it holds dependent instructions in ID for one or more cycles, as shown for I3 in Figure 2, at the cost of a small throughput penalty.

Branch Prediction: for control hazards, a branch predictor guesses the outcome of a conditional branch (e.g., "temperature exceeds limit") before it is resolved, so fetch can continue speculatively down the predicted path. A simple 2-bit saturating-counter dynamic predictor is well suited here because threshold-check branches in sensor loops are usually strongly biased one way (most readings are within normal range), giving high prediction accuracy with modest hardware cost. On misprediction, the few speculatively fetched instructions are flushed and fetch restarts from the correct target - a small, bounded penalty appropriate for a real-time system where predictable worst-case latency matters.

6. Superscalar Processing, Out-of-Order Execution, Speculative Execution and SMT (CO3)

Beyond a single 5-stage pipeline, several established techniques can further raise throughput. Each comes with a power and design-complexity cost, which matters for an energy-constrained industrial node, so the choice must be justified rather than assumed.

Technique	How It Increases Throughput	Suitability for This Industrial Monitoring Processor
Superscalar Execution	Fetches and issues more than one instruction per cycle to multiple parallel execution units	Useful in moderation (2-issue) - sensor routines mix independent ALU ops (scaling multiple channels) that can issue together
Out-of-Order (OoO) Execution	Reorders ready instructions ahead of stalled ones using a reservation station / reorder buffer, hiding latency	Adds significant hardware and power cost; only a light OoO window is justified, since workloads are short and mostly predictable
Speculative Execution	Executes instructions past an unresolved branch based on a prediction, committing results only if correct	Helpful for throughput but must be bounded and used cautiously - deep speculation increases worst-case latency variance, which conflicts with real-time determinism
Simultaneous Multithreading (SMT)	Interleaves instructions from two or more hardware threads in the same pipeline to fill idle issue slots	Well suited here - separate hardware threads can service independent sensor channels concurrently, keeping the pipeline busy during memory-access stalls on any one channel

Given the requirement for predictable real-time behaviour and low power, the recommended configuration is a modest 2-issue superscalar pipeline with a shallow out-of-order window (enough to hide typical cache-miss latency but not so deep that worst-case timing becomes hard to bound), light/bounded speculative execution restricted to a small number of in-flight speculative instructions, and 2-way SMT so that the same core can service two sensor channels in parallel and keep functional units busy. Full desktop-class deep OoO and heavy speculation are deliberately avoided, since their power and design cost outweigh their benefit for this workload.

7. Hierarchical Memory System (CO4)

Sensor-processing workloads repeatedly re-read recent samples (temporal locality) and scan through contiguous sample buffers (spatial locality), which makes a cache-based memory

hierarchy very effective. Figure 3 shows the proposed hierarchy, moving from the fastest and smallest level (CPU registers) down to the slowest and largest (secondary/flash storage used to log historical data).

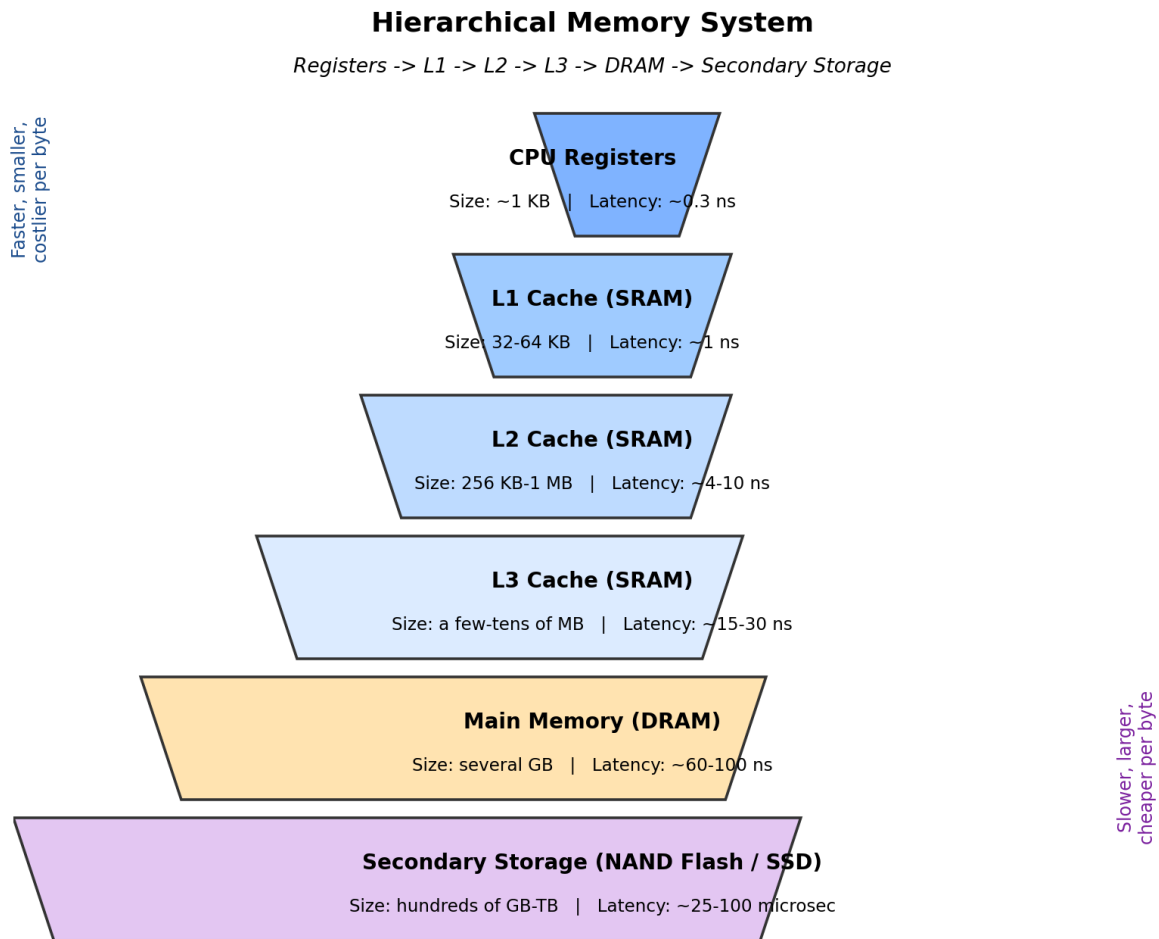


Figure 3: Proposed hierarchical memory system, with representative size and latency at each level.

7.1 Cache Levels

L1 cache (32-64 KB, split I/D, ~1 ns access) sits closest to the CPU and holds the current sensor routine's instructions and the samples it is actively working on. L2 cache (256 KB-1 MB, ~4-10 ns) is shared or private per core and catches the working set that overflows L1 - for example, buffers for several sensor channels being processed in the same control loop. L3 cache (a few to

tens of MB, ~15-30 ns), if present, is shared across cores and absorbs cross-channel and cross-task reuse, such as historical samples referenced by a trend/anomaly-detection routine.

7.2 Virtual Memory and Paging

Even an embedded monitoring processor benefits from virtual memory: it lets the operating/RTOS layer isolate the memory space of different tasks (e.g., the sensor-acquisition task and the alarm-logging task) so a fault in one cannot corrupt another, and it allows a uniform address space regardless of how physical DRAM is populated. Paging divides virtual and physical memory into fixed-size pages (e.g., 4 KB); a page table (accelerated by a small TLB - translation look-aside buffer) maps virtual pages to physical frames. A TLB hit adds negligible latency, while a TLB miss requires a page-table walk; because the monitoring task's working set is small and stable, a modest TLB (32-64 entries) achieves a very high hit rate in practice.

7.3 Average Memory Access Time (AMAT)

Cache effectiveness is commonly summarized using AMAT:

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

Worked example: suppose L1 has a hit time of 1 ns and a miss rate of 5%, and an L1 miss is serviced by L2 with an access time of 8 ns and an L2 miss rate of 20%, with an L2 miss finally serviced by main memory (DRAM) at 80 ns.

$$\text{AMAT}(\text{L2}) = 8 \text{ ns} + (0.20 \times 80 \text{ ns}) = 8 + 16 = 24 \text{ ns}.$$

$$\text{AMAT}(\text{L1, overall}) = 1 \text{ ns} + (0.05 \times 24 \text{ ns}) = 1 + 1.2 = 2.2 \text{ ns}.$$

So the effective memory access time seen by the processor is about 2.2 ns per access, roughly 36x faster than always going to DRAM (80 ns) and about 2400x faster than a direct flash access - demonstrating why the cache hierarchy is essential rather than optional for keeping the pipeline fed.

7.4 Cache Hit/Miss Rate

If, over a monitoring interval, the L1 cache services 950,000 accesses out of 1,000,000 total references directly, then: Hit Rate = $950,000 / 1,000,000 = 95\%$, and Miss Rate = $1 - 0.95 = 5\%$,

matching the figure used above. This hit rate is realistic for sensor-buffer processing because the same small set of channel buffers is revisited every control-loop iteration.

8. Memory Technology Comparison (CO4)

Choosing the right technology at each level of the hierarchy - and for persistent data logging - directly affects both performance and the energy-efficiency goal central to this assignment. The table and chart below compare SRAM, DRAM, NAND Flash, MRAM, and RRAM on the dimensions that matter for an industrial monitoring processor.

Technology	Typical Latency	Volatility / Endurance	Relative Power	Best Role in This Design
SRAM	~1 ns	Volatile, unlimited endurance	Higher static power (6-transistor cell)	L1/L2/L3 cache - needs to be fast above all else
DRAM	~60-100 ns	Volatile, needs refresh, unlimited endurance	Moderate, but refresh adds standby power	Main memory - large, reasonably cheap working store
NAND Flash	~25-100 microsec (page)	Non-volatile, limited write endurance ($\sim 10^3$ - 10^5 cycles)	Low idle power, higher write energy	Secondary storage / historical data logging
MRAM	~10 ns	Non-volatile, very high endurance	Low standby power (no refresh needed)	Persistent working registers / fast non-volatile buffer for power-loss-safe logging
RRAM	~5 ns (varies by cell)	Non-volatile, good endurance, still maturing	Very low standby power	Emerging option for low-power embedded cache or non-volatile scratchpad

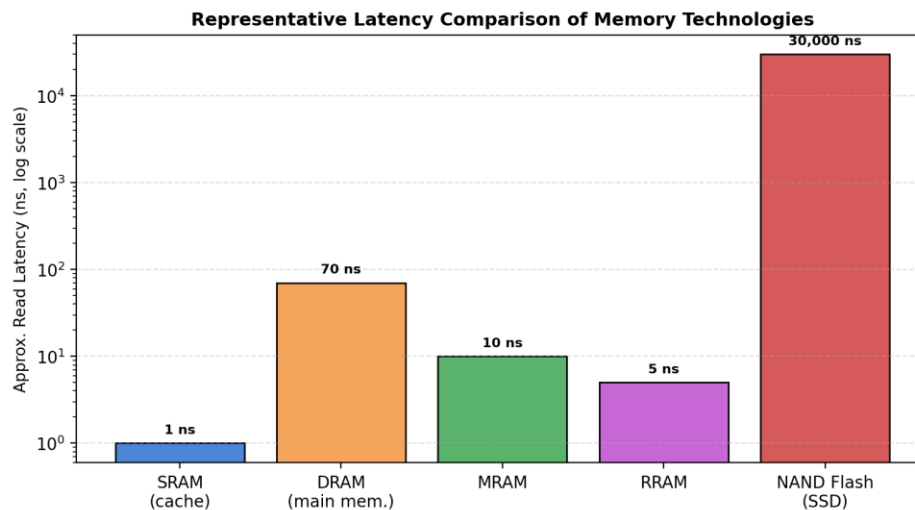


Figure 4: Representative read-latency comparison across memory technologies (log scale).

SRAM and DRAM remain the right choice for the cache hierarchy and main memory respectively because of their speed, but their volatility means all data is lost on power interruption - a real risk on a factory floor. Emerging non-volatile technologies (MRAM, RRAM) close this gap: they are fast enough to sit close to the CPU yet retain data without power, which is directly useful for safely buffering the last few sensor readings and alarm state across a brief power dip, without the write-endurance concerns of NAND Flash.

9. Results, Comparison and Engineering Recommendation

9.1 Summary of Results

- Unpipelined CPI for the representative sensor routine: 2.45 cycles/instruction; execution time 24.5 microsec at 1 GHz.
- With 5-stage pipelining, ideal throughput approaches 1 instruction/cycle once the pipeline is full, i.e. close to a 2-2.5x speed-up on the same clock, minus a small penalty from load-use stalls and occasional branch mispredictions.
- AMAT with the proposed 3-level cache hierarchy: 2.2 ns, versus 80 ns if every access went straight to DRAM - roughly a 36x reduction in effective memory latency.
- Cache hit rate for the sensor-buffer working set: 95% at L1, which is what makes the AMAT figure achievable in practice.

9.2 Recommended Processor and Memory Organization

Based on the analysis above, the recommended design for the industrial-monitoring processor is: a 5-stage pipelined core with forwarding and a 2-bit dynamic branch predictor to minimize stall and flush penalties; a modest 2-issue superscalar front end with a shallow out-of-order window and bounded speculation, combined with 2-way SMT so one physical core can service two sensor channels concurrently; and a three-level cache hierarchy (private L1 I/D-cache, private or shared L2, shared L3 if multiple cores are used) backed by DRAM main memory, with an MRAM or RRAM buffer reserved for the last few readings and alarm/state variables so they survive brief power interruptions without the write-endurance limits of flash; and NAND Flash reserved purely for bulk historical data logging, where its slower access time is acceptable.

This combination directly targets the three original design goals: real-time responsiveness (pipelining plus a bounded, predictable hazard-handling strategy), throughput sufficient for

multiple concurrent sensor streams (superscalar issue and SMT), and low energy consumption (a right-sized, non-over-engineered core; a cache hierarchy that sharply cuts the number of costly DRAM/flash accesses; and non-volatile MRAM/RRAM that removes the standby refresh power DRAM would otherwise spend on data that changes rarely).

9.3 Bottlenecks Identified

- Load-use data hazards remain the dominant pipeline stall source; deeper forwarding paths or reordering loads earlier in code can reduce this further.
- L2/L3 miss penalty (24 ns effective) still dominates AMAT on the (rare) L1-miss path; a larger or more associative L2 would reduce this at an area/power cost that would need to be re-justified against the energy budget.
- NAND Flash write endurance is the long-term reliability bottleneck for continuous data logging; wear-levelling and use of MRAM/RRAM for the highest-churn data mitigates this.

10. Supporting Tool / Simulation Evidence

The performance figures in Sections 4 and 7 were derived analytically using the standard CPI, execution-time, and AMAT formulas from computer-architecture theory, applied to a representative sensor-processing instruction mix and a representative cache configuration; the accompanying diagrams (Figures 1-4) were generated to visualize the processor organization, the pipeline hazard/stall behaviour, the memory hierarchy, and the relative latency of memory technologies. For a lab/simulation extension of this assignment, the same instruction mix can be re-run on a cycle-accurate simulator (e.g., gem5 or a simple 5-stage pipeline simulator such as WepSIM/MARS/Ripes) to confirm the hand-calculated CPI and stall counts, and the resulting trace/output should be committed to the course GitHub repository together with this report, per Deliverable 5.

GitHub Repository Link: github.com/kanishk_xp/CSA1203-CA

11. Reflection: Design Decisions, SDG Relevance and Learning Outcomes

11.1 Justification of Design Decisions

Every major choice in this design was made by weighing performance against power and predictability rather than maximizing raw throughput. A full 5-stage pipeline was chosen over a simpler single-cycle design because the throughput gain is large relative to its modest added control complexity. Superscalar issue, out-of-order execution and speculation were deliberately kept shallow/bounded rather than desktop-class, because for a real-time industrial controller, predictable worst-case latency and low idle power matter more than peak throughput. Similarly, the memory hierarchy favours a hit-rate-driven cache design plus non-volatile MRAM/RRAM for critical state, rather than simply maximizing cache size, because the goal is to cut the number of expensive DRAM/flash accesses while protecting essential data from power loss - not to build the fastest possible cache regardless of cost.

11.2 Connection to the Sustainable Development Goals

SDG 7 (Affordable and Clean Energy): the entire design - the right-sized pipeline, bounded speculation, a cache hierarchy that sharply reduces slow DRAM/flash accesses, and non-volatile memory that removes DRAM refresh power - is aimed at cutting the energy each processing node consumes, which matters directly when hundreds of such nodes are deployed across a factory.

SDG 9 (Industry, Innovation and Infrastructure): an efficient, real-time-capable monitoring processor is exactly the kind of infrastructure that lets a manufacturing facility modernize (Industry 4.0) - catching faults earlier, reducing downtime, and enabling data-driven process improvement.

SDG 12 (Responsible Consumption and Production): lower power consumption per node reduces the facility's overall energy footprint; using non-volatile memory to protect state and log data reliably also reduces wasted rework/scrap that would otherwise result from an undetected fault or a lost sensor reading during a brief power interruption, supporting more responsible use of materials and energy in production.

11.3 Challenges and Learning Outcomes

The most challenging part of this assignment was reasoning about trade-offs rather than simply describing techniques - for example, recognizing that speculative execution and deep out-of-order

windows, while good for average-case throughput, can work against the predictability that a real-time industrial controller needs, and that the "best" choice therefore depends on the application rather than being universally true. Working through the CPI, AMAT, and hit-rate calculations by hand also made the abstract formulas concrete: it is one thing to know the AMAT formula, and another to see how a 95% L1 hit rate turns an 80 ns DRAM access into an effective 2.2 ns access. Overall, this assignment reinforced how processor and memory design decisions in computer architecture are never made in isolation - they must be justified together against a specific application's performance, power, and reliability requirements.